OATHFORGE

The Living Oath

Biblia de diseño, expansión y revisión técnica

Una campaña cooperativa persistente, juramentos con consecuencias y un mundo que recuerda. Diagnóstico del código, propuestas de sistemas y más de cien ideas y variantes para Aetherfall.

Preparado para
Josue Rolando Naranjo



Oathforge 0.7.6 | 14 octubre 2026 | Base: 31c166f4c2d2

Índice

1. Diagnóstico: un juego pequeño con una identidad fuerte 5

2. Qué existe realmente y qué está pendiente 6

3. Qué aprender de otros juegos 7

4. Conservar, modificar, cambiar y retirar 8

5. Bucle de juego y experiencia de Telegram 9

6. Clases: identidad antes que cantidad 10

7. Especializaciones: 32 conceptos, una entrega pequeña 11

8. Linajes, culturas y trasfondos 12

9. Combate que funciona en un chat 13

10. Cooperativo: la siguiente prueba del producto 14

11. Profesiones y maestrías: ampliar lo que ya funciona 15

12. Recursos y geografía con propósito 16

13. Economía y comercio sin inflación gratuita 17

14. Equipo, loot y procedencia 18

15. Juramentos: el sistema que justificaría el nombre 19

16. Facciones y relaciones que no son barras de puntos 20

17. Misiones: más verbos, menos cadenas de matar 21

18. Mundo vivo, eventos y campañas 22

19. Endgame, gremios y PvP 23

20. Programación: lo que estás haciendo bien 24

21. Programación: hallazgos y correcciones prioritarias 25

22. Programación: cooperativo, persistencia y mantenibilidad 26

23. Arquitectura objetivo y contratos 27

24. IA: libertad controlada, memoria y coste 28

25. Verificación, operaciones y documentación 29

26. Catálogo de ideas adicionales: exploración y narrativa 30

27. Catálogo de ideas adicionales: comunidad y economía 31

28. Catálogo de ideas adicionales: combate y futuro 32

29. Evidencias y referencias 33

30. Anexo: verificación ejecutada 34

OATHFORGE

The Living Oath

Biblia de diseño, expansión y revisión técnica

Documento de decisiones y propuestas para convertir el prototipo de Aetherfall en un RPG persistente que se juega desde Telegram. Preparado para Josue Rolando Naranjo.

Base revisada: Oathforge 0.7.6, rama main, commit 31c166f4c2d2e75b8051595f1e5b260f354340f5. **Fecha:** 14 de octubre de 2026. Las cifras y recomendaciones de balance son hipótesis de diseño salvo que se identifiquen como datos del repositorio o resultados de pruebas.

Lectura recomendada: capítulos 1-4 para decidir el rumbo; 5-19 para diseñar sistemas y contenido; 20-25 para programarlos; catálogo final para elegir futuras expansiones. El documento Roadmap convierte estas decisiones en entregas ejecutables.

1. Diagnóstico: un juego pequeño con una identidad fuerte

Oathforge ya tiene algo más valioso que una lista extensa de clases: un circuito jugable en el que el jugador actúa, el motor resuelve, la IA narra y los personajes recuerdan. Su mejor dirección es un RPG cooperativo persistente para grupos pequeños, con un mundo compartido que cambia de forma limitada y comprensible. El término MMO describe la aspiración; todavía no describe la implementación multijugador.

La promesa que propongo es: **tus juramentos tienen consecuencias, tus aventuras dejan huellas y tus amigos pueden intervenir en ellas**. La diferencia frente a un incremental de texto no debería ser conseguir más objetos por minuto, sino tomar decisiones que conecten combate, reputación, artesanía y mundo.

La versión revisada presenta una vertical slice individual: ocho clases, combate d20 por turnos, cadena de misiones, recolección temporizada, fabricación, tiendas, memoria de NPC y rumores. Las tablas de grupo anticipan el cooperativo, pero el servicio de combate aún sincroniza y recompensa a un solo personaje. Hay mucho contenido declarativo preparado y menos funcionalidad de grupo de la que podría sugerir el nombre del proyecto.

Mi recomendación principal: terminar la experiencia de niveles 1-5, poner a dos amigos en la misma cripta y conseguir que el servidor sobreviva a un reinicio durante esa aventura. Después activar especializaciones, encargos y un evento que cambie Crownreach. Eso prueba la identidad completa sin necesitar veinte regiones.

Objetivos de producto propuestos: sesiones activas de 10-25 minutos, actividades de campo que pueden reclamarse al volver, grupos iniciales de 2-4 personas y una campaña que permite pausar. Estos valores son objetivos de prueba, no mediciones de tus jugadores.

Tres pilares: decisiones con consecuencias legibles; cooperación accesible sin horarios obligatorios; resolución auditable independiente de la IA. Cada feature nueva debe reforzar al menos uno y no romper los otros.

2. Qué existe realmente y qué está pendiente

El inventario siguiente se obtuvo del código y del contenido, no solo del README. No confundir una columna, una tabla o un nombre de especialización con un sistema terminado.

Sistema	Evidencia de la versión revisada	Decisión
Personaje	8 clases, 4 linajes, 4 trasfondos; compra de puntos; un personaje por usuario	Conservar y pulir la creación
Progresión	Nivel máximo 5; umbrales 0/200/500/1000/1800 XP	Mantener hasta cerrar el cooperativo
Especializaciones	32 nombres en classes.yaml; campo specialization	Implementar reglas y elección, no solo mostrar nombres
Combate	Iniciativa, filas, acción/bonus/movimiento, estados, consumibles	Conservar; generalizar el servicio
Mundo	3 zonas y 17 lugares declarados; grafo de rutas	Expandir por regiones con propósito
Misiones	6 definiciones; objetivos, acciones y efectos	Añadir vías alternativas y contratos repetibles
Criaturas	10 definiciones en monsters.yaml	Corregir referencias documentales a 9
Economía	62 objetos; compra/venta NPC; reputación afecta precios	Medir producción y consumo antes de mercado
Oficios	7 oficios con plaza y cocina libre; 2 plazas activas	Conservar dos plazas
Recolección	Minería, tala, hierbas, pesca, rastreo y desuello	Progreso libre separado de oficios
Fabricación	25 recetas; ingredientes/resultado/XP declarativos	Balancear y agregar pedidos
Memoria	Recuerdos NPC y relación por personaje	Separar hechos, declaraciones y resúmenes
Mundo compartido	world_facts y rumores; dos facciones	Convertir un hecho en una consecuencia visible
Grupos	Party, PartyMember y EncounterParticipant	Estructura preparada; gameplay pendiente
Interfaz	Español/inglés; mapa, fichas, diario, informes	Reducir ruido y aclarar errores

Fuentes internas: content/*.yaml; engine/progression.py; database/models.py; services/combat_service.py; services/professions.py; services/activities.py. El README conserva descripciones anteriores de seis oficios y doce recetas, y lista profesiones como pendiente pese a que existen. planning/roadmap.md contiene candidatos de versiones anteriores a la actual. La documentación necesita una única foto del estado real.

No existe una carpeta /docs en el árbol revisado. Los documentos antiguos están en planning/ y planning/referencias/. Estos dos PDF pueden servir como documentos principales de diseño; conservar README, CHANGELOG, instrucciones de colaboración y referencias históricas necesarias.

3. Qué aprender de otros juegos

Esta comparación usa fuentes públicas oficiales cuando están disponibles. Las adaptaciones son propuestas originales para Oathforge, no reglas que debas copiar literalmente. No se ha jugado ni auditado el código de esos productos.

Referente	Hallazgo documentado	Adaptación propuesta
D&D	Pruebas d20, ventaja/desventaja y separación de combate, exploración e interacción [S1]	Tirar cuando el fallo tenga una consecuencia; permitir resolver una misión sin combatir
WoW: profesiones	Encargos y especialización permiten distinguir artesanos [S2]	Pedido con ingredientes y comisión en custodia; reputación de fabricante
WoW: Warbands	Progreso y colecciones compartidos entre personajes [S3]	A largo plazo, códice y cosméticos por cuenta; juramentos por personaje
Chat Wars	Canal oficial muestra castillos, gremios y batallas programadas; también cambios revertidos y fixes [S4]	Comunidad y preparación colectiva, con conflictos optativos y reglas simples
SimpleMMO	Interacciones breves, misiones, mercado, gremios y jefes compartidos [S5]	Un menú claro para volver al juego; profundidad detrás de pocas acciones

D&D: conservar la claridad causal entre acción y resultado. Evitar importar todas sus excepciones. Un chat sufre si cada ataque abre cinco interrupciones y tres ventanas de reacción. Usar filas, condiciones acotadas y reacciones preseleccionadas.

WoW: tomar identidad de roles, habilidades reconocibles, jefes anunciados y artesanos útiles. Evitar tareas diarias obligatorias, una escalada indefinida de equipo y el catálogo masivo antes de conocer el coste de balancearlo. Los encargos son especialmente adecuados para tus dos plazas de oficio.

Chat Wars: el diseño social de un bot sí puede sostener organización, recursos y guerras. Su historial oficial también muestra que la posesión territorial y sus permisos generan muchas ramas de reglas. Además, las novedades de agosto visibles en el canal se anuncian para staging: no tratarlas como confirmación de producción [S4]. Empezar con un proyecto comunitario, no con conquistas persistentes.

SimpleMMO: tomar la accesibilidad, no el volumen de objetos. Su presentación oficial enumera más de ocho mil piezas de loot [S5]; Oathforge puede conseguir más decisiones con veinte objetos que interactúan con escenas que con cientos de espadas de estadísticas casi idénticas.

Conclusión de diseño: una capa ligera de MMO sobre una campaña cooperativa. El mundo común ofrece mercado, rumores y proyectos. Las aventuras instanciadas protegen el ritmo y la coherencia de cada grupo.

4. Conservar, modificar, cambiar y retirar

Decisión	Elemento	Por qué y cómo
Conservar	Motor sin llamadas externas	Permite reproducir reglas y probar balance
Conservar	YAML validado y textos es/en	Agregar contenido sin dispersar strings por handlers
Conservar	Dos oficios; recolección libre	Crea identidad sin obligar a depender de otros para todo
Conservar	Linajes clásicos y ocho clases	Ya permiten variedad suficiente para una beta
Conservar	Reputación y memoria NPC	Es la semilla de consecuencias persistentes
Modificar	Descanso largo de 80 minutos	Probar una recuperación activa accesible y otra offline; mostrar exactamente qué restaura
Modificar	Linaje humano: +1 a seis atributos	Comparar su valor total con bonos y rasgos de los demás; migrar con compensación
Modificar	Inmunidad enana al envenenamiento	Valorar resistencia o ventaja limitada para que el peligro siga importando
Modificar	Dificultad sugerida por IA	Derivarla de la escena y reglas; la IA solo propone el enfoque
Modificar	Venta de joyería al NPC	Limitar demanda o margen para no crear una fábrica infinita de oro
Cambiar	Confianza modificada por respuesta IA	Política determinista de interacción social y evidencias
Cambiar	Combate centrado en p1	Participantes identificados y autoridad por usuario/actor
Cambiar	Presupuesto de IA comprobar/luego registrar	Reserva atómica antes de llamar; liquidación posterior
Retirar	Candidatos obsoletos del Roadmap	Un backlog vigente; historial separado
Retirar	Tiradas sin consecuencias	Resolver automáticamente lo evidente; fallo significativo cuando se tira
Posponer	Guerras territoriales, multiclase, raids grandes	Multiplican permisos, balance y coordinación
Posponer	Mini App, TTS de cada mensaje, agentes autónomos	No solucionan todavía el bucle cooperativo

Retirar significa retirar del diseño objetivo o del backlog inmediato; este informe no ha borrado archivos ni cambiado reglas. Antes de eliminar contenido jugable, medir uso, ofrecer migración y explicar la decisión a los testers.

Regla de selección: primero corregir un bloqueo del jugador; después ampliar una decisión que ya disfruta; finalmente introducir un sistema nuevo. La lista de ideas de este documento es una reserva de posibilidades, no una obligación de implementarlas todas.

5. Bucle de juego y experiencia de Telegram

Primeros diez minutos: /start; elección guiada de clase; atributos recomendados editables; llegada a Crownreach; conversación breve con Bram; una decisión de ruta; una prueba con resultado explicado; combate corto; recompensa; aprender que el diario conserva lo ocurrido. La creación avanzada debe ser opcional: elegir 27 puntos antes de entender el juego puede expulsar a un principiante.

Bucle activo: identificar un problema, prepararse, viajar, resolver una escena, recibir consecuencias y decidir qué hacer después. Cada regreso a la ciudad debe traer una razón: encargo, información, fabricación, descanso, conversación o cambio de reputación.

Bucle offline: elegir una actividad, conocer duración y riesgo, dejar al personaje trabajando y reclamar al volver. No introducir un castigo por reclamar tarde. Una actividad resuelta debe entregar el mismo resultado tras un reinicio. No convertir todas las aventuras en una cuenta atrás.

Bucle social: publicar una necesidad, invitar a un amigo, compartir una aventura, fabricar algo útil para él y dejar una noticia en la taberna. Las relaciones deberían servir incluso si solo hay cinco jugadores activos.

Interfaz propuesta

Un panel de estado principal: lugar, PV, estado actual, grupo y tres acciones prioritarias. Botones secundarios para inventario, mapa, diario y personaje. Editar el panel cuando cambia la escena; enviar un mensaje nuevo para un desenlace importante. El informe completo de tiradas debe seguir disponible, aunque la vista normal muestre solo el resumen.

Ejemplo de resultado: “Percepción: dados 7 y 16; se conserva 16 por ventaja. +4 = 20 contra dificultad 15. Encuentras una puerta, pero abrirla exige una llave o forzarla”. Es más útil que una tirada abreviada sin explicar qué cambió.

Añadir /resume, /party y /unstuck con significado acotado. /unstuck reconstruye el panel desde estado persistente; no teletransporta ni restaura recursos gratuitamente. /resume presenta hechos recientes y la siguiente decisión pendiente. La preferencia “resumen / detalle” evita imponer informes largos a todos.

Implementación: separar renderizadores de personaje, escena, comercio y combate; cada vista recibe un DTO listo para presentar. Mantener comprobación de permisos en servicios. Los callbacks deben usar identificadores breves: Telegram limita callback_data a 1-64 bytes [T1]. Probar el tamaño codificado, no el número de caracteres.

6. Clases: identidad antes que cantidad

Mantener las ocho clases actuales. Cada una necesita una decisión distintiva por turno, una aportación fuera de combate y una debilidad comprensible. No es necesario que todas hagan el mismo daño ni que puedan curar igual; sí que todas puedan completar el inicio individual y sean deseables en un grupo.

Clase	Identidad propuesta	Decisión distintiva	Utilidad narrativa
Paladín	Protector que paga por su juramento	Gastar poder en castigo o protección	Negociar bajo palabra, reconocer profanaciones
Pícaro	Preparación y oportunidades	Crear apertura o aprovecharla	Infiltrar, falsificar, desactivar
Guerrero	Control fiable del frente	Interceptar, empujar o rematar	Liderazgo militar, resistencia física
Sacerdote	Sostener sin borrar el peligro	Curar ahora o prevenir daño	Ritos, duelo, refugio
Mago	Control del espacio y recursos	Daño inmediato o condición	Identificar magia, resolver sellos
Druida	Adaptación del grupo al entorno	Forma, control o restauración	Rutas naturales y trato con fauna
Bardo	Manipular oportunidades	Inspirar aliado o perturbar rival	Rumores, pactos y actuación
Brujo	Poder condicionado por un pacto	Marcar un enemigo o defenderse	Pactos, secretos y conocimiento prohibido

Carga inicial: tres botones principales por clase y una opción de detalle. No mostrar diez habilidades con usos, enfriamientos y costes a nivel 1. Las habilidades pasivas deben explicarse cuando afectan una tirada.

Roles flexibles: “frente / apoyo / control / daño” como capacidades, no como una composición obligatoria. Una cripta para dos jugadores debe funcionar sin sanador. Recuperación de emergencia, cobertura y consumibles permiten configuraciones distintas. Las recompensas no deberían exigir haber curado ni haber infligido cierto porcentaje de daño.

Balance propuesto: evaluar victoria, PV restantes, recursos gastados, rondas y acciones disponibles. Comparar contra varios enemigos y secuencias de combates con descanso limitado. Una tasa alta contra un enemigo aislado no demuestra balance de campaña.

Clases posteriores: Explorador primero, porque conecta arco, rastreo y supervivencia. Monje después si existe un motor de movilidad útil. Artífice como clase solo cuando se distinga de Artificiería como oficio. No agregar Bárbaro antes de demostrar que se diferencia del Berserker del Guerrero.

7. Especializaciones: 32 conceptos, una entrega pequeña

Los nombres siguientes ya existen en classes.yaml. Las mecánicas descritas son propuestas. Recomiendo implementar inicialmente **una especialización funcional por clase**, dejando las demás en el catálogo de diseño. Después agregar una segunda por clase según preferencias observadas. Mostrar opciones aún no jugables como futuras, sin permitir seleccionarlas.

Clase	Especializaciones existentes y dirección de diseño
Paladín	Vengeance: perseguidor de un objetivo; Protection: interceptar un golpe; Redemption: reducir daño y negociar; Inquisition: revelar defensas mágicas
Pícaro	Assassin: preparación de remate; Spy: información y debilitamiento; Duelist: ventaja en duelo; Trapper: trampas con cargas
Guerrero	Champion: consistencia y críticos; Battle Master: maniobras; Guardian: proteger la fila; Berserker: riesgo a cambio de presión
Sacerdote	Life: curación eficiente; Light: daño radiante y limpieza; War: apoyo ofensivo; Grave: impedir una caída y tratar no muertos
Mago	Pyromancer: daño prolongado; Cryomancer: ralentizar y cubrir; Arcanist: eficiencia de recursos; Chronomancer: manipular iniciativa con límites
Druida	Shapeshifter: formas; Warden: raíces y defensa; Summoner: una criatura acotada; Spores: desgaste y zonas
Bardo	Illusionist: señuelos; Diplomat: herramientas sociales; War Chanter: apoyo de fila; Trickster: desviar una amenaza
Brujo	Fiend: poder y coste; Necromancer: un servidor; Hexblade: marca cuerpo a cuerpo; Void: control con contraprestación

Paquete mínimo por especialización: una pasiva, una habilidad activa, una interacción narrativa y una misión de desbloqueo. No crear un árbol de quince nodos para cada opción. A nivel 5 la especialización debe cambiar la manera de jugar, no añadir solo +1 a una tirada.

Ejemplo: Guardián Sagrado. Pasiva: el primer aliado protegido recibe una reducción limitada de daño. Activa: interceptar un ataque de un aliado en la misma fila, una vez por combate. Interacción: asumir una deuda o garantizar un acuerdo. Misión: proteger a un testigo sin entregarlo a ninguna facción. Son propuestas a validar; no están en el motor actual.

Ejemplo: Criomante. Pasiva: una aplicación de frío mejora defensa o control, sin acumular indefinidamente. Activa: congelar una ruta durante una ronda, con salvación. Interacción: atravesar un río creando un paso que dura poco. Evitar negar turnos repetidamente a un jefe.

Contrato técnico: specialization_key; requisitos; efectos tipados; coste; cooldown; targeting; descripción es/en; versión de reglas. ability_defs, class_passives, full_resources y los renderizadores deben usar un catálogo efectivo que combine clase y especialización. No editar solo un campo del personaje.

Respecialización: en ciudad, confirmación y coste moderado; conservar progreso. Bloquear cambios durante combate, actividad o transacción de equipo. No permitir entrar con recursos de una especialización y cambiar para obtener otra reserva completa.

8. Linajes, culturas y trasfondos

Conservar humano, elfo, enano y orco, pero separar **linaje biológico, cultura y oficio previo**. Así un orco puede ser erudito y un elfo puede ser minero sin estar penalizados por su identidad. Los rasgos deben abrir posibilidades y no decidir qué clase es “correcta”.

El humano recibe actualmente +1 a seis atributos; los otros reciben tres puntos y rasgos. Esto no prueba por sí solo que esté desbalanceado, porque importan modificadores, competencias y encuentro, pero exige comparación. La inmunidad enana al estado envenenado también debe probarse frente a escenarios relevantes, no solo frente al bandido.

Linaje nuevo propuesto	Fantasía	Rasgo jugable acotado	Vínculo de mundo
Ashborn	Marcados por una tormenta arcana	Reducir una vez daño de fuego; deja señal visible	Supervivientes de una catástrofe
Tidekin	Pueblos de riberas y mareas	Ventaja limitada en travesía acuática	Stormport y Silverbrook
Hollowbound	Viven con un eco funerario	Percibir una pista de espíritu por escena	Cripta y Hollow Choir
Ironwrought	Constructos con voluntad	Resistir una condición a cambio de mantenimiento	Talleres de Frosthold
Thornblood	Afinidad con bosque antiguo	Paso natural o reconocer una planta extraña	Thornwood y Verdant Pact
Smallfolk	Pequeños viajeros ingeniosos	Repetir una prueba concreta por descanso	Caravanas y rutas

Añadir primero dos, después de revisar los cuatro actuales. Evitar que todos los nuevos sean resistentes al daño: el motor necesita rasgos sociales, de exploración y de información.

Culturas propuestas: Crownreach Citizen, Frosthold Artisan, Thornwood Exile, Stormport Sailor, Border Caravan. Pueden dar idioma, contacto, receta común o rumor inicial. Los bonos permanentes de combate siguen un presupuesto común.

Trasfondos propuestos: Cartógrafo, Minero, Cazador, Mensajero, Médico de campaña, Aprendiz de escriba, Noble caído, Superviviente de la cripta, Desertor, Cocinero de caravana, Contrabandista y Acólito disidente. Cada uno aporta dos ganchos, una competencia y una elección inicial; no doce misiones exclusivas.

Implementación: validadores de referencias y límites, rasgos tipados en engine, migración para personajes existentes y una opción gratuita de revisar atributos afectados por el cambio. Los textos culturales nunca deben afirmar que todos los miembros de un linaje tienen la misma moralidad.

9. Combate que funciona en un chat

Las filas actuales son una buena abstracción: posición útil sin necesidad de un tablero. Expandir primero su significado. Vanguardia protege rutas cuerpo a cuerpo; retaguardia facilita disparos; cobertura es una propiedad de escena limitada. Un tercer eje de altura puede existir como etiqueta excepcional, sin convertir el motor en una cuadrícula.

Añadir gradualmente: proteger aliado, ayudar una prueba, empujar de fila, romper cobertura, interrumpir un ritual y retirarse organizadamente. Cada opción debe competir con el ataque, no añadir una acción gratuita adicional.

Reacciones: preselección “interceptar primer golpe / conservar / reaccionar a conjuro”. El motor ejecuta la regla cuando corresponde. Evitar solicitar un mensaje a cada defensor en cada golpe. Para PvP, publicar la política antes del turno del rival y fijarla hasta la siguiente ronda.

Jefes por fases: anuncio, oportunidad de respuesta y resolución. Ejemplo: el Guardián levanta su sello y marca la vanguardia; los jugadores pueden cambiar de fila, destruir un foco o defenderse; en su siguiente turno golpea esa fila. El daño no debería aparecer sin advertencia salvo que esa sorpresa sea una mecánica enseñada.

Tres encuentros propuestos: patrulla con arquero que necesita cobertura; ritual con objetivo de interrumpir tres focos; criatura hambrienta que puede distraerse con comida. Estos encuentros reutilizan el motor y prueban si las clases aportan más que daño.

Inteligencia enemiga determinista: prioridades declarativas por arquetipo. Guardián protege; depredador persigue heridos; cultista canaliza; arquero busca cobertura; capitán cambia orden. El comportamiento actual toma el rival de menor PV y el primer ataque adecuado. Mejorarlo con prioridades y puntuaciones probables, manteniendo RNG inyectable. No llamar a Gemini para escoger cada golpe.

Derrota: rescate o retirada con coste, pérdida parcial limitada de suministros y una cicatriz narrativa opcional. Evitar pérdida permanente del personaje en la beta. **Huida:** por individuo en un grupo; no terminar toda la batalla porque uno escapa. **Caído:** estado recuperable antes de derrotado; introducirlo solo cuando la curación a aliados exista.

Implementación: efectos discriminados, límites de acumulación, duración explícita y objetivo validado. Agregar una prueba de que los enemigos no atacan a un jugador que ya huyó, que una cura no resucita sin permiso y que ninguna cadena de control bloquea indefinidamente al jefe.

10. Cooperativo: la siguiente prueba del producto

Empezar por dos personas en una aventura instanciada, ampliable a cuatro. Un chat de grupo muestra escena e iniciativa; la ficha, inventario y secretos quedan en privado. El idioma del grupo es configurable y las comunicaciones privadas conservan el idioma del usuario.

Invitar: token corto, caducidad, invitador y estado del grupo. Aceptar verifica identidad Telegram, plaza, estado libre y requisitos. **Preparar:** todos ven destino, duración esperada, dificultad y reparto. **Comenzar:** un líder propone; los demás confirman preparación. No obligar al grupo a compartir todos los avances individuales.

Turnos rápidos: plazo inicial propuesto de 90 segundos; una advertencia; defensa automática si se agota y el usuario aceptó esa política. **Campaña lenta:** plazo de 6-24 horas elegido por el grupo. No mezclar ambas velocidades en el mismo combate. Los plazos son parámetros de prueba, no requisitos universales.

Ausencias: defender un turno, pausar o retirarse; expulsión fuera de combate. Si el líder sale, transferencia de liderazgo por una regla publicada. Dos ausencias seguidas pueden ofrecer pausa en lugar de consumir automáticamente recursos. No controlar libremente el personaje de un amigo.

Botín inicial: recompensas individuales equivalentes y objetos de misión compartidos por instancia. Aplazar necesidad/codicia hasta que haya equipo raro y suficiente demanda. Los materiales pueden repartirse en rotación definida al comenzar; conservar cada reparto en el registro.

Exploración: el líder propone destino y los miembros confirman; votación solo para decisiones importantes. Un integrante no puede activar combate mientras otro comercia. El estado de grupo debe definir cuáles acciones individuales siguen disponibles.

Cambios necesarios: active_encounter debe buscar por participante; start_encounter construye todos los combatientes; act resuelve usuario a actor; _sync_and_maybe_finish persiste PV y recursos de todos; recompensas se pagan con claves únicas por encuentro y personaje. El motor ya recibe actor_id, pero el servicio fija p1 en múltiples rutas.

Orden de bloqueos: adoptar un único orden para operaciones de encuentro, grupo y personajes; bloquear personajes por id ascendente. Todos los caminos, incluidos handlers y trabajos de timeout, deben respetarlo. No añadir locks distintos según quién actúa.

11. Profesiones y maestrías: ampliar lo que ya funciona

Mantener dos plazas de oficios de producción y recolecciones libres. Cocina es una afición en professions.yaml; pesca funciona como maestría, no como una profesión de fabricación allí. Esta distinción debe verse claramente en /professions.

Oficio actual	Primera expansión útil	Especialización futura
Herrería	Escudo, herramienta y reparación	Armero / Forjador de armas
Alquimia	Antídoto y preparación contra un jefe	Restauración / Reactivos
Carpintería	Arco, kit de campamento, mobiliario	Arquería / Construcción
Peletería	Bolsa y equipo de exploración	Armaduras / Equipo de campo
Joyería	Gemas situacionales y encargos	Engastes / Reliquias
Escribanía	Mapas y sellos de contrato	Runas / Cartografía
Artificiería	Herramientas y bombas con límites	Dispositivos / Demoliciones
Cocina libre	Menú de viaje y banquete de grupo	Recetas regionales

Maestrías por uso: los rangos actuales 0-5 son adecuados. Añadir una elección a rango 3: minería profunda o prospección; tala eficiente o rarezas; hierbas abundantes o plantas difíciles; pesca segura o trofeos; rastreo rápido o presa especial; desuello de calidad o cantidad. Una elección modifica la actividad, no solo multiplica todos sus premios.

Calidad: dos calidades al comienzo, corriente y cuidada. No introducir tres calidades de materia prima por cada hierba antes de que el inventario las pueda explicar. Una calidad cuidada puede alterar duración, fiabilidad o una propiedad secundaria; evitar que sea siempre daño mayor.

Descubrimiento: receta común del maestro; receta regional de misión; receta excepcional de jefe con protección frente a mala suerte. Duplicados pueden volverse páginas intercambiables cuando el mercado esté listo.

Dependencias entre oficios: permitir ingredientes comunes vía NPC y reservar componentes especiales para cooperación. Un jugador no debe quedarse bloqueado por haber elegido dos oficios que requieren un tercer artesano inexistente.

Tres plazas: no recomiendo abrirlas ahora. Evaluar cuando existan varias recetas útiles por oficio y encargos completados por distintos jugadores. Una tercera plaza podría desbloquearse mediante una misión, pero reduce especialización económica: medir su efecto.

Implementación: skill_xp, profesión activa y recetas aprendidas separados; fabricación en lote con cantidad limitada y validación antes de consumir. El servicio actual craft es una buena base atómica; añadir coste monetario o herramienta solo a través de efectos tipados y bajo el mismo lock.

12. Recursos y geografía con propósito

La Mina del Paso Viejo, la Arboleda Antigua, el Claro de la Raíz Roja y Silverbrook ya permiten producir materiales. Su expansión debe añadir decisiones geográficas: recurso, peligro, información y acceso. No añadir lugares cuyo único cambio sea una descripción más larga.

Mina: veta segura común; túnel profundo con riesgo anunciado; hallazgo “algo brilla” convertido en descubrimiento real; reparación comunitaria que abre una veta temporal. **Bosque:** ciclo de plantas, huellas raras y permiso de tala del Verdant Pact. **Ribera:** cebos, clima simplificado y una criatura que altera la pesca. **Prado:** caza por rastreo con identificación de presa; dejar huir a un animal puede afectar un encargo.

Los recursos compartidos finitos crean competencia, pero también permiten monopolizar el juego cuando hay pocos jugadores. Empezar con disponibilidad personal y proyectos globales. Introducir agotamiento compartido solo en un nodo especial, con regeneración anunciada y reserva accesible para novatos.

Clima: dos o tres estados por región, con reglas declaradas y duración persistida. Lluvia puede mejorar hierbas y dificultar la travesía; niebla altera rastreo; helada abre una ruta. No usar una API meteorológica real: Aetherfall debe tener su propia coherencia y reproducibilidad.

Herramientas: primero requisitos y mejoras de actividad; después durabilidad si añade decisiones. Reparaciones previsibles, sin romper herramientas de forma aleatoria. Mostrar coste antes de expedición. No castigar al recién llegado con una herramienta inutilizable sin oro para arreglarla.

Expedición larga: elegir una estrategia de regreso antes de salir: cargar más, ir sigiloso o llevar escolta. El riesgo y el premio deben derivarse de reglas. La crónica cuenta lo ocurrido después; no decide si encontró una gema.

Datos nuevos: gathering nodes con resource_table, danger, duration, access_rule, depletion_policy y content_version. Persistir la versión de tabla con la actividad: actualmente la semilla se guarda, pero claim lee el contenido y rango vigentes. Una actualización durante la espera puede cambiar su distribución. Elegir explícitamente si se usa la foto al iniciar o las reglas al reclamar; recomiendo la foto al iniciar para expectativas estables.

13. Economía y comercio sin inflación gratuita

La economía necesita tres flujos diferenciados: oro creado por el sistema, oro transferido entre jugadores y oro destruido mediante servicios. Un mercado no destruye dinero salvo sus tasas. Vender materiales al NPC sí crea oro; fabricar y vender sin límite puede convertirse en la mejor actividad universal.

Inventario de fuentes actuales: recompensas de monstruos, misiones y venta de objetos; también robos a NPC. **Sumideros actuales visibles:** compras, descanso y cambio de oficio. **Riesgo observado:** npcs.steal no comprueba una bolsa agotada ni un enfriamiento. Repetir la interacción podría crear oro; el rate limit de interfaz no convierte ese recurso en finito. Confirmarlo con una prueba específica y fijar cooldown o stock por periodo.

Sumideros propuestos: reparación de herramienta, viaje seguro, comisión de encargo, aprendizaje de receta avanzada, mejora de sede, identificaciones y cosméticos. No cobrar por cada botón ni hacer que el descanso básico sea inaccesible.

Libro de economía: registrar causa, origen, destino, cantidad, personaje y clave de operación. Para oro creado, origen sistema; para destruido, destino sistema. Suma semanal creada menos destruida, oro por jugador activo y velocidad de circulación. No necesitas blockchain ni una contabilidad distribuida.

Pedidos antes que subasta: un cliente deposita ingredientes y comisión; un artesano acepta; craft usa componentes reservados; el resultado pasa al cliente; oro al artesano; tasa al sistema. Cancelación y caducidad devuelven la reserva una sola vez. El artesano no puede quedarse con ingredientes ni entregar otro objeto.

Mercado inicial: listados a precio fijo, pocos filtros, duración, custodia y confirmación. Usar precio entero, qty positiva y estado listed/sold/cancelled/expired. Bloquear listing, compradores y vendedores en orden fijo. Los objetos equipados, vinculados o de misión no se publican.

Balance: calcular margen de recetas con precios vigentes y bonificación de reputación extrema; detectar ciclos comprar-fabricar-vender rentables sin límite. El tiempo invertido puede producir beneficio, pero debe estar presupuestado y comparado con misiones. Joyería puede ser rentable sin dominar todas las rutas de progreso.

Propuestas iniciales, no cifras definitivas: tasa de mercado de 3-5%, límite pequeño de anuncios y demanda NPC diaria para joyería. Validar con simulación antes de aplicarlas.

14. Equipo, loot y procedencia

ItemInstance ya tiene equipped_slot, crafted_by e history. Mantener esos campos y convertir la procedencia en algo visible: “forjada por Mira del grupo”, “obtenida en la apertura del osario”, “reparada tras la crisis del hierro”. El objeto es una historia, no solo un número.

Ranuras iniciales: arma, mano secundaria, armadura y accesorio. Mostrar comparación antes de equipar. Expandir a herramientas y un objeto de utilidad cuando haya interfaz clara. Evitar quince ranuras con diferencias mínimas.

Rarezas propuestas: común, cuidada, singular y reliquia. La rareza expresa posibilidades y procedencia; no una escalera inevitable de estadísticas. Una antorcha singular revela runas; una cuerda de calidad permite un rescate; una reliquia ofrece un coste narrativo.

Afijos limitados: máximo uno en el primer sistema. Ejemplos: resistente al barro, consagrado contra no muertos, silencioso para infiltración, imantado para prospección. Cada afijo requiere un efecto tipado y una descripción de su límite. Nada de texto libre IA que el motor intente interpretar como un poder.

Sets: primero dos piezas y un efecto situacional. No dar +20% universal por llevar cinco piezas que obliguen a descartar cualquier hallazgo interesante. **Encantamiento:** escoger entre alternativas, con vista previa; evitar destrucción permanente por fallo en la beta.

Protección de mala suerte: tras varias victorias sin un componente, incrementar o garantizar su entrega. Guardar contador por personaje y tabla. No garantizar siempre el mejor objeto en cada visita; dar progreso hacia una recompensa evita que la cripta se sienta inútil.

Loot cooperativo: material personal y una recompensa elegida al finalizar. Los objetos únicos globales son una expansión posterior: pueden excluir a amigos que juegan tarde. Para reliquias relevantes, varias copias o una custodia de grupo con consentimiento explícito.

Implementación: separar objetos apilables sin identidad de instancias con atributos y procedencia; no mezclar cantidades cuando crafted_by, afijos o vínculo difieren. Índices y constraints para ranuras y cantidad; historial de cambios acotado o tabla de eventos para no inflar JSON indefinidamente.

15. Juramentos: el sistema que justificaría el nombre

Hacer del juramento un sistema transversal. No reservarlo solo al paladín: cualquier personaje puede comprometerse con una persona, facción o proyecto. El paladín tiene capacidades particulares para sostenerlo, pero un pícaro también puede prometer que nadie morirá en un robo.

Estructura: ofrecido, aceptado, activo, cumplido, roto, reparado. Cada juramento incluye sujeto, condiciones observables, plazo si existe, consecuencias anunciadas y vías de reparación. El jugador debe conocer las reglas antes de aceptar.

Juramento propuesto	Condición evaluable	Consecuencia y reparación
Guard the Witness	Testigo llega vivo a un destino	Confianza y acceso; si falla, rescate o restitución
No Blood for Iron	Resolver contrato minero sin matar humanos	Título y apoyo del pacto; compensación a afectados
Keep the Ember	Mantener un refugio durante evento	Beneficio comunitario; reconstrucción si cae
Return What Was Taken	Devolver objeto por instancia	Reputación y memoria; recuperar un equivalente
Never Leave One Behind	No abandonar aliado caído en expedición	Reconocimiento del grupo; misión de rescate
Speak Before Steel	Intentar negociación válida antes de combatir	Nuevo acceso social; penitencia breve si se rompe

Lo difícil: “no mentir” o “ser honorable” no tienen un detector objetivo robusto. Empezar con promesas que el motor puede evaluar. Las promesas libres escritas por el jugador pueden existir como roleplay sin recompensas automáticas hasta formalizarlas en condiciones tipadas.

Implementación: oath_definitions YAML; character_oaths con definición/version/status/counters; suscripción a eventos canónicos. Al ejecutar quest_completed, ally_rescued o npc_harmed, actualizar el juramento en la misma transacción. Una clave por evento evita contabilizar dos veces.

La IA: ofrece la redacción y recuerda el compromiso; no decide por impresión moral si lo rompiste. Puede explicar una evidencia ya confirmada. Si la situación es ambigua, mantener el juramento pendiente y permitir una resolución diseñada, no una sanción inventada.

Valor narrativo: un título o una cicatriz cambia cómo te saluda un NPC. No todos los juramentos deben dar poder de combate. Los mejores conectan relaciones y rutas futuras.

16. Facciones y relaciones que no son barras de puntos

Conservar Silver Covenant y Crimson Syndicate. Añadir facciones cuando cada una sostenga al menos una misión, un servicio y una tensión con otra. Diferenciar reputación pública, confianza personal y favores pendientes.

Facción propuesta	Interés	Servicio	Conflicto útil
Verdant Pact	Mantener el equilibrio de Thornwood	Rutas y hierbas raras	Tala intensiva y expansión minera
Hollow Choir	Custodiar voces de la cripta	Ritos, investigación y sellos	Reliquias vendidas al Sindicato
Ironbound Consortium	Producción y seguridad del mineral	Encargos y herramientas	Huelga y recursos del bosque
Wayfarers' League	Rutas abiertas y viajeros	Escoltas, mapas y refugios	Peajes y control territorial
Ashen Collegium	Entender anomalías arcanas	Identificación y recetas	Experimentos que dañan el entorno
Crown Remnants	Recuperar símbolos del viejo reino	Historia, títulos y archivos	Legitimidad del gobierno actual

Escalas: reputación de -100 a 100 puede seguir, pero presentar rangos y requisitos: desconocido, aceptado, confiable, aliado. Evitar un descuento por cada punto si el jugador no entiende su valor. El precio actual usa una fórmula continua; puede conservarse internamente mientras la interfaz muestra su efecto.

Favores: un NPC puede deberte ayuda concreta. Gastar un favor consigue una pista, un componente o un refugio, nunca permiso de administrador. La deuda queda registrada y se consume una vez.

Consecuencias compatibles: ayudar al pacto puede cerrar un encargo de tala, pero abrir una ruta alternativa. Evitar que una primera elección bloquee definitivamente media campaña. Las decisiones fuertes pueden ser reversibles mediante acciones costosas, no mediante un botón gratuito.

Relaciones de grupo: definir si el NPC conoce a cada personaje o al grupo. No copiar confianza de un miembro a todos. Una negociación del líder solo compromete recursos compartidos aprobados.

Implementación: reglas de reputación por evento; umbrales con desbloqueos declarativos; memory.source_event_id y nivel de certeza. Los resúmenes del modelo no son evidencia de que un jugador pagó una deuda. Conserva el evento de pago y genera la memoria a partir de él.

17. Misiones: más verbos, menos cadenas de matar

El sistema actual de etapas y efectos es una buena base. Expandir con objetivos que reconozcan transporte, investigación, fabricación, conversación y ayuda. Cada misión debería declarar quién tiene autoridad sobre su progreso: personaje, grupo, instancia o mundo.

Plantilla de misión: problema; dos soluciones; requisito observable; coste de cada solución; consecuencia local; vínculo con una facción; una pista opcional; recompensa concreta. No generar cuarenta encargos si todos piden cinco pieles con una narración distinta.

Seis aventuras propuestas

The Iron That Whispers. Una veta responde a nombres de muertos. Minar acelera la ciudad, sellarla calma la cripta, investigarla da una receta. Reutiliza mina, osario, gremio y memoria de NPC. Finales distintos alteran un nodo de recursos temporal.

The Witness at Silverbrook. Un mensajero pide escolta y el Sindicato lo busca. Ruta corta con riesgo, ruta lenta por el bosque o refugio en capilla. El jugador decide revelar una carta; la confianza depende de eventos registrados.

The Hollow Feast. Una taberna necesita un banquete para negociar. Cocina, pesca y diplomacia contribuyen; se puede comprar lo común y producir lo especial. El combate es opcional si alguien sabotea la entrega.

A Crown Without a King. Reliquia con varias interpretaciones. Los NPC ofrecen relatos contradictorios y el archivo conserva evidencias. No pedir a la IA que invente retrospectivamente cuál era la verdad.

The Road That Forgot Us. Una ruta desaparece por niebla. El grupo crea un mapa con tres pistas y puede rescatar una caravana. El explorador ayuda, pero todas las clases tienen un camino.

The Debt of Old Tusk. Alguien quiere matar a la criatura y otro conservarla. Cazar, trasladar o negociar un recinto tienen materiales, pruebas y efectos diferentes. La elección vuelve a aparecer en un contrato futuro.

Fallos que avanzan: una tirada fallida revela una pista incompleta, aumenta el coste o provoca una escena, en lugar de bloquear la misión hasta mañana. La restricción actual de repetir habilidad por lugar impide insistir, pero puede limitar enfoques distintos; usar challenge_id y approach permitido para separar repetir el mismo intento de intentar otra solución.

Generación procedural: primero plantillas con objetivos y referencias válidas. La IA puede redactar el encargo aprobado; no crear objetos, XP ni condiciones arbitrarios en producción. Una misión generada se valida y tiene un presupuesto antes de publicarse.

18. Mundo vivo, eventos y campañas

Empezar con un solo evento que atraviese sistemas: **La crisis del hierro**. Una vía minera se derrumba; recolectores aportan madera, artesanos fabrican soportes, aventureros despejan túneles y diplomáticos negocian turnos de trabajo. Todos aportan sin necesitar el mismo horario.

Fases: anuncio, preparación, contribución, resolución y epílogo. Umbrales se ajustan al tamaño de comunidad observado, no a una población imaginaria de miles. El evento puede terminar con éxito parcial; no exigir que todos jueguen diariamente.

Efectos persistentes acotados: un precio cambia durante unos días; se abre una ruta; queda una estatua o rumor; aparece una receta. No destruir Crownreach porque dos amigos no llegaron a tiempo.

Hechos y rumores separados: world_fact canónico registra que el túnel se reparó; un rumor puede afirmar quién fue el héroe, incluso de forma incompleta. El motor consulta el hecho para abrir la ruta. Un NPC consulta conocimiento y rumores para hablar. Añadir visibilidad y origen para no filtrar secretos privados en la taberna global.

Periódico Herald of Aetherfall: resumen de hechos confirmados con nombres públicos de personaje. Para el primer sistema, redactar con plantilla o generar borrador validado. Publicación mediante outbox; una caída de Telegram no revierte el evento ni lo publica repetidamente.

Temporadas: arcos de 4-8 semanas como hipótesis organizativa, sin reset de personaje. Una temporada cambia el problema del mundo y conserva un epílogo. Las temporadas de PvP, cuando existan, pueden reiniciar ranking y dar cosméticos; no borrar amistades, oficios y juramentos.

Regiones futuras: Frosthold para forja y clanes; Stormport para comercio y piratería; Ashen March para anomalías; Hollow Depths para investigación funeraria. Cada región necesita su recurso, dilema, jefe, servicio, tres tipos de misión y retorno a regiones anteriores.

Implementación: world_events con estado/version/inicio/fin; contribution con unique(event_id, operation_id); reducers deterministas; scheduler persistente. Los efectos sobre tiendas y rutas usan modificadores temporales, no edición directa de YAML durante el evento. Un administrador puede pausar sin inventar una resolución.

19. Endgame, gremios y PvP

El endgame inicial puede existir en nivel 5: especialización, misiones ramificadas, cripta con modificadores y proyectos artesanales. No es necesario subir a nivel 20 para mantener interés. Aumentar el límite cuando haya contenido que cambie decisiones, no para alargar una barra.

Endgame cooperativo: expedición de tres escenas con recursos limitados; jefe con dos fases; objetivo de rescate; reliquia elegida y un cambio narrativo. Dos modificadores bastan: oscuridad y ritual, por ejemplo. Combinaciones deben validarse para no anular todas las habilidades de una clase.

Gremios pequeños: nombre, emblema, tablón, proyecto y permisos de banco. No pedir diez miembros para que funcione si el juego comienza entre amigos. La sede primero es un proyecto narrativo, después ofrece estaciones y depósitos; no dar multiplicadores permanentes que vuelvan irrelevante a quien juega solo.

Roles: líder, oficial y miembro inicialmente; tesorería como permiso separado cuando haya banco. Cualquier retiro registra quién, cuánto y por qué. El liderazgo no concede propiedad automática de los objetos de los demás.

Duelos: consentimiento de ambos, reglas visibles y estado separado. Restaurar la foto de recursos al finalizar; bloquear transferencias durante el duelo. Sin pérdida de equipo. El ranking llega después de probar clases y emparejamiento.

Arena 2v2: recompensa cosmética y límites de participación puntuable contra la misma pareja. Detectar arreglos no significa sancionar automáticamente: registrar y revisar. Los duelos entre amigos siguen disponibles sin puntuar.

Conflicto de facciones: primero una competición de contribuciones con varias vías. Luego una batalla optativa por un objetivo temporal. Posponer territorio permanente, asedios y guerra abierta: requieren población, horarios, moderación y equilibrio de bandos.

Colecciones: recetas, bestiario, canciones, títulos, cicatrices y mapas. Son progreso horizontal compatible con pausas. Un álbum de aventuras debería conservar las decisiones y compañeros, no solo el contador de victorias.

Monetización futura: si hace falta cubrir hosting e IA, priorizar apoyo voluntario y cosméticos. Nunca vender autoridad narrativa, mejores tiradas ni saltos de reglas. Esta es una recomendación de producto, no una evaluación legal o comercial.

20. Programación: lo que estás haciendo bien

La arquitectura tiene decisiones adecuadas para el tamaño actual. No recomiendo reescribirla en microservicios, cambiar de lenguaje ni reemplazar PostgreSQL. El trabajo consiste en cerrar contratos y generalizar puntos concretos.

Buena decisión	Evidencia	Cómo conservar su valor
Motor separado	engine/combat.py, dice.py, gathering.py reciben RNG	Mantener fuera SQL, Telegram y Gemini
Servicios transaccionales	session_scope y locks de personajes/encuentros	Toda nueva escritura conserva invariantes
Control optimista	version_id_col en Character y Encounter	Validar versión de la decisión, además del flush
Idempotencia de botones	audit.claim y clave única; combat callback version/turn	Extender a texto, scheduler y economía
Contenido declarativo	schemas.py y loader.py	Agregar relaciones y límites validados
Fallbacks	NullLLM y catálogo validado	Probar cada resultado y modo sin proveedor
Actividades persistentes	WorldActivity guarda ends_at, seed y result	Persistir también la foto de reglas y notificaciones
Fabricación	Ingredientes verificados bajo lock; rollback si desaparecen	Mantener resultado tipado y pago atómico
Diagnóstico	CombatAction, recorder, informes y simulador	Añadir correlation_id y confirmar commit
Localización	Catálogos es/en	Comprobar claves y variables en cada release
Caché de medios	file_id por bot y checksum	No re-subir retratos innecesariamente

Matiz: motor “determinista” significa reproducible con las mismas entradas y secuencia RNG, no que el juego carezca de azar. Para reproducir una partida completa también hacen falta versión de reglas y contenido, estados iniciales y orden de operaciones. Guardar solo un texto de tirada no equivale a guardar todo el replay.

Matiz sobre versiones: SQLAlchemy detecta cambios concurrentes de filas versionadas durante flush; no valida por sí solo que una intención interpretada diez segundos antes siga siendo semánticamente válida [T2]. Hace falta expected_version y revalidación.

Escala recomendada: monolito modular, un proceso de bot al comienzo y workers acotados si aparecen tareas. PostgreSQL puede sostener este alcance sin introducir Redis, Kubernetes o un bus distribuido como requisito inicial.

21. Programación: hallazgos y correcciones prioritarias

Los siguientes puntos proceden de lectura del código. “Observado” significa que el patrón existe; no implica que se haya explotado en un servidor real. Las pruebas específicas indicadas son la forma de confirmar su efecto.

P0 - Autoridad y duplicación

A. Confianza y memoria decididas por IA. NpcReply permite trust_delta de -2 a 2. actions.talk guarda memory_note y aplica el delta directamente. Contradice la regla fuerte de que la IA nunca decide resultados cuando la confianza habilita consecuencias. Corrección: reply solo entrega diálogo y propuestas; social_rules calcula confianza por eventos/tiradas y límites; guardar “el jugador afirma X” separado de “X ocurrió”. Prueba: diez mensajes que pidan amistad o inventen un pago no pueden producir confianza o hechos canónicos automáticamente.

B. Presupuesto no reservado. Budget.allow consulta totales, _generate llama al proveedor y Budget.record suma tras éxito. Varias llamadas pueden pasar el mismo límite. El lock de record no protege allow. La primera creación de una fila también puede competir. Corrección: fila diaria global y por usuario, UPSERT y reserva condicional de capacidad en transacción corta; liquidar tokens al terminar. Los intentos fallidos deben registrarse como intentos aunque no produzcan uso confirmado. Prueba: veinte solicitudes con un cupo restante conceden solo la capacidad autorizada.

C. Texto sin idempotency_key. execute_free_text usa audit.log, no claim, y on_text no suministra update_id como identidad de operación. Locks serializan pero no impiden ejecutar dos veces el mismo mensaje. Corrección: operation_id basado en bot/update y estado persistente accepted/resolved/failed. Prueba: repetir el mismo update cien veces paga una vez; dos mensajes distintos siguen siendo acciones distintas.

D. Contexto obsoleto. La ruta normal ejecuta con in_combat previo, pero location es opcional y no se pasa habitualmente; tampoco transporta versión del personaje/encuentro. Corrección: ActionContext con context_version, location_key, encounter_id y turn_index; comprobar bajo lock y reinterpretar o rechazar si cambió. Prueba: mover al personaje durante la interpretación no ejecuta una acción social en el lugar anterior.

E. Robo como fuente repetible. npcs.steal tira y crea oro sin agotamiento de bolsa ni cooldown visible en ese servicio. Corrección: stock o recompensa limitada por NPC/periodo y condición social; no permitir recuperar reputación por conversación ilimitada para robar otra vez. Prueba: una secuencia repetida tiene producción monetaria acotada.

22. Programación: cooperativo, persistencia y mantenibilidad

P0 antes de grupos

Identidad de actor. active_encounter filtra character_id del propietario y act, build_context y sincronización usan p1. Generalizar por EncounterParticipant; comprobar que effective_user controla el personaje y el combatant_id del turno. Nunca confiar en actor_id recibido en callback.

Pertenencia exclusiva. PartyMember solo asegura unicidad del par grupo/personaje; no de personaje entre grupos. Añadir restricción para un grupo activo, o membership con estado y unique parcial. Una reserva de expedición evita que un personaje entre simultáneamente en actividad individual y aventura grupal.

Expiración de turno. Persistir deadline, estado y version. Un job ejecuta compare-and-swap del turno; si el jugador ya actuó, no hace nada. La memoria de JobQueue sirve para despertar, no como fuente de verdad. Un reinicio reconstruye pendientes desde PostgreSQL.

P1 - Corregir costes y coherencia

Crónica duplicada. chronicles.tell revisa texto antes de llamar y bloquea solo después. Dos solicitantes pueden generar dos textos y gastar dos llamadas, aunque se guarde uno. Añadir estado generating con lease y request_id o single-flight persistente. No mantener un lock durante Gemini.

Notificación de actividad. schedule_ready_ping crea un job en memoria; _ready_ping busca actividad por personaje y no por id. La recompensa se conserva en BD, pero el aviso no queda reconstruido automáticamente en el código revisado. Guardar activity_id y una entrega única; recuperar pendientes al arrancar.

E/S durante transacción. Algunos caminos de callbacks llaman send_photos con session abierta; _act también envía en una salida temprana con el personaje bloqueado. Preparar una vista y enviar tras commit. Una llamada lenta a Telegram no debe prolongar un lock de gameplay.

Registro previo a commit. recorder.battle se llama dentro de la resolución. Si luego hay rollback, el archivo podría describir un cambio no confirmado. Marcar eventos tentativos o publicar el registro desde outbox después de commit. El log SQL confirmado sigue siendo la autoridad.

Restricciones BD. Revisar gold >= 0, qty > 0, límites de PV compatibles con reglas y unicidad de equipo por ranura. Validar en servicio y BD; no añadir constraints que impidan una transición legítima. AiUsage con user_id NULL requiere atención: una UNIQUE ordinaria no trata múltiples NULL como el mismo usuario global.

P2 - Hacer fácil ampliar

render.py tiene alrededor de 58 KB y callbacks.py alrededor de 21 KB. Separarlos por dominio cuando se toque cada feature, evitando una refactorización masiva sin beneficio. La importación local de activities desde combat_service evidencia un ciclo entre servicios; eventos de fin de combate pueden desacoplarlo.

Las dependencias tienen límites inferiores amplios. Guardar un lock reproducible y CI con PostgreSQL. El paquete busca content/ junto al código: comprobar instalación como wheel fuera del checkout e incluir recursos si falla. No asumir que instalar editable verifica distribución.

23. Arquitectura objetivo y contratos

Flujo de acción: Telegram recibe operación; aplicación identifica usuario; carga foto corta; parser interpreta si hace falta; servicio bloquea y valida versión/permiso; engine resuelve con RNG; transacción guarda estado/eventos/outbox; commit; render y narración entregan resultado. Una nueva feature debe encajar aquí antes de añadir su menú.

ActionRequest propuesto: operation_id, user_id, character_id, verb, target_id, expected_character_version, expected_encounter_version, content_version, payload tipado. user_id y character_id se obtienen del servidor, no del texto del jugador. **ActionResult:** outcome_id, ok, error_code, events, committed_versions, narrative_facts y next_view.

Contrato del intérprete: propone verb/target/approach usando ids ofrecidos. No entrega oro, XP, daño, confianza ni inventario. **Contrato del narrador:** recibe hechos aprobados y puede expresarlos; sus datos no vuelven al reducer. **Contrato del diálogo:** línea, intención social propuesta y recuerdos candidatos con procedencia. **Contrato del generador de misión:** borrador offline validado; nunca habilitarlo directamente desde el chat sin revisión automática de reglas y presupuesto.

Dato nuevo	Campos mínimos	Invariante
operation	key, actor, request, result, status	Resultado económico único por key
domain_event	id, type, aggregate, version, payload, cause	Orden por agregado y payload versionado
outbox	event_id, destination, payload, attempts, next_try	Reintentos sin volver a resolver gameplay
ai_reservation	request_id, day, user, model, tokens, state, lease	Reserva antes de la llamada
encounter deadline	encounter, actor, deadline, version	Solo expira el turno esperado
reward_grant	encounter, character, reward_type	Una entrega por participante y tipo
craft_order	cliente, recipe, fee, escrow, status	Ingredientes y resultado en custodia
oath	character, definition, version, status, counters	Estado derivado de eventos válidos

No migrar todo a event sourcing ahora. Mantener modelos actuales como estado operativo y añadir eventos de dominio donde hagan falta para auditoría, consecuencias y outbox. Un bus interno puede ser una lista de eventos procesados en la transacción; no necesita otro servidor.

Escalado: el primer proceso puede serializar updates, pero jobs, herramientas y futuros workers ya introducen concurrencia. Diseñar reservas y autoridad en BD evita depender de esa serialización accidental. Los locks de PostgreSQL se mantienen hasta fin de transacción y pueden producir deadlocks si los órdenes difieren [T3].

24. IA: libertad controlada, memoria y coste

Mantener Gemini como proveedor, respetando las instrucciones actuales del proyecto. Una interfaz LLM ya permite desacoplar tareas sin migrar de proveedor. No recomendar dos o tres modelos por moda: elegirlos tras medir latencia, coste y calidad por función. Los límites del proveedor cambian; los valores locales de config.py no garantizan cuota disponible.

Reducir llamadas: botones deterministas sin parser; intérprete sencillo de verbos frecuentes cuando el modelo falla; narración breve del resultado; crónica bajo petición o al final; resúmenes por lote. La IA aporta libertad y voz, pero no necesita describir cada compra de una cuerda.

Modelo de coste: llamadas por sesión = interpretaciones + narraciones + diálogos + crónicas + resúmenes. Una acción libre puede consumir dos llamadas; una conversación iniciada mediante parser puede añadir otra. Ejemplo hipotético: 20 amigos × 15 acciones libres × 2 = 600 llamadas, antes de conversaciones y crónicas. No es una predicción de uso ni un precio de API.

Memoria en tres capas: episodio canónico con event_id; relación numérica calculada; resumen semántico de episodios. Añadir declaraciones del jugador como categoría distinta. Al resumir, conservar ids de evidencia, intervalo y versión. Una frase inventada no debe desplazar el hecho de un robo.

Recuperación: filtro por NPC/personaje y visibilidad; priorizar relevancia, importancia y recencia; limitar tokens; impedir que una noticia privada aparezca en rumor global. Los cinco recuerdos actuales son una buena primera aproximación, pero pueden dejar una escena reciente fuera si existen muchos recuerdos antiguos importantes.

Seguridad semántica: delimitadores de player_text ayudan, pero no garantizan que el modelo ignore instrucciones maliciosas. La defensa real es el contrato tipado, allowlist y validación del servidor. También memoria_note, nombres y rumores son entradas potencialmente hostiles para futuros prompts.

Narración verificable: resultado mecánico visible siempre. Para hechos sensibles -muerte, objeto, oro, juramento, puerta abierta- usar frases o campos controlados. Puede comprobarse que las entidades y recompensas mencionadas pertenezcan al resultado; si no, caer a plantilla. Un segundo LLM no es necesario para cada turno y tampoco garantiza verdad.

Indicadores: uso por purpose/model, llamadas intentadas y exitosas, tokens, reserva, latencia p95, fallbacks y rechazos. Definir día de presupuesto con zona explícita; hoy date.today y day_start dependen de hora local del servidor.

25. Verificación, operaciones y documentación

Una suite que pasa sobre SQLite no verifica SELECT FOR UPDATE ni carreras de creación de filas en PostgreSQL. Conservar los tests rápidos y exigir una ejecución PostgreSQL para entregas de economía, cooperativo y presupuestos. Usar una BD de prueba aislada; nunca apuntar tests destructivos a producción.

Batería necesaria: duplicación de update, doble callback, dos compras del mismo listing, turno frente a timeout, reinicio durante actividad, crónica simultánea, un cupo IA disputado, invitación vencida, jugador ajeno al grupo y migración desde datos anteriores. Comparar estado final y eventos, no solo el mensaje mostrado.

Balance: fijar seed y contenido; evaluar clases contra enemigo único, grupos, jefe y ruta completa. Medir victoria, duración, consumo y contribución. El simulador actual sirve para combate individual; extenderlo a grupo y a decisiones de apoyo antes de concluir que todas las especializaciones están balanceadas.

Operación online: un servicio para bot y un PostgreSQL con persistencia, secretos en configuración, migraciones controladas y backups automáticos. No se propone plataforma ni precio sin investigación específica. Probar restauración: un backup que nunca se ha restaurado no demuestra recuperabilidad.

Objetivos propuestos: restaurar una beta en menos de una hora y perder como máximo un día de progreso si solo hay backup diario. Ajustarlos si se implementan intercambios valiosos. Alertas para errores persistentes, BD inaccesible, presupuesto al 80% y ausencia de procesamiento de deadlines.

Registros: correlation_id por acción; retirar tokens y claves de cualquier captura; retención limitada para conversaciones y prompts; acceso administrativo acotado. Mantener eventos económicos más tiempo que narraciones de debug. recorder guarda texto y nombres: documentarlo a los testers y ofrecer borrado de sus datos.

Documentación: estos PDF son diseño principal. README sigue describiendo instalación y estado actual; CHANGELOG sigue el historial; TODO de testers conserva bugs; planning puede quedar como fuentes editables y decisiones breves. No sustituir instrucciones operativas por una biblia de cuarenta páginas.

Migración recomendada de documentos: agregar docs/Oathforge_The_Living_Oath.pdf y docs/Oathforge_Roadmap.pdf; actualizar enlaces; mover referencias anteriores a archivo si ya no son normativas; preservar trazabilidad. Estos entregables no modifican el repositorio remoto ni eliminan planning/.

26. Catálogo de ideas adicionales: exploración y narrativa

Las ideas siguientes complementan los sistemas desarrollados antes. “Pronto” significa después de los bloqueos técnicos; “después” requiere cooperativo estable; “futuro” requiere evidencia de uso. Cada fila añade una vía concreta de implementación.

Idea	Implementación inicial	Momento
Pistas privadas	clue_grants por personaje y escena; compartir con consentimiento	Después
Puertas con varias soluciones	challenge_id y resoluciones por llave/fuerza/ritual	Pronto
Mapas incompletos	Rutas descubiertas en flags tipados; compra de fragmentos	Pronto
Rescate de caravanas	Instancia con progreso y estado de escoltados	Después
Campamento	Gastar kit; escoger descanso, guardia o preparación	Después
Idiomas de mundo	Etiqueta de conocimiento abre una pista alternativa	Después
Fama local	Contadores por zona, independientes de facción	Después
Alias de infiltración	Identidad de escena con caducidad, no otra cuenta	Futuro
Archivo de evidencias	Diario relaciona clue_id con quest_id	Pronto
Juicio en ciudad	Tres evidencias y una defensa; efectos deterministas	Después
Pactos temporales	Contrato con condición/evento y coste visible	Después
Marcas de maldición	Condición persistente curable por misión	Después
Sueños de personaje	Escenas sin recompensa económica; hechos etiquetados	Futuro
Misterio comunitario	Aportaciones de pistas, una verdad diseñada	Después
Fantasmas de expedición	Leer replay de una aventura antigua en el lugar	Futuro
Diario compartible	Exportación seleccionada sin secretos ni ids Telegram	Después
Mentor NPC	Consejos por fallos observados, sin modificar dados	Pronto
Encuentros de camino	Tabla con combate, comercio, ayuda y descubrimiento	Pronto
Objetos de utilidad	Efectos de cuerda, antorcha, kit y mapas sobre challenges	Pronto
Destinos del trasfondo	Una escena corta por trasfondo reutilizando rutas	Después

Paquete pequeño recomendado: puerta de osario con tres soluciones, evidencia persistente y objeto de utilidad. Usa el contenido existente, enseña agencia y puede probarse individualmente antes del grupo.

Criterio editorial: cada pieza de lore debe explicar una relación, peligro, elección o deseo. Descripciones de quinientas palabras no mejoran un menú de Telegram. Entregar un párrafo evocador y permitir “Saber más”.

27. Catálogo de ideas adicionales: comunidad y economía

Idea	Implementación inicial	Momento
Contratos de caza	Plantilla, target, plazo y premio presupuestado	Pronto
Encargos artesanales NPC	Demanda diaria acotada por receta y región	Pronto
Lotes de fabricación	qty limitada; coste total prevalidado	Pronto
Reforja	Elegir propiedad secundaria conservando procedencia	Después
Desmantelar equipo	Tabla fija de retorno; sin ciclos de beneficio	Después
Banco personal	Transferencia atómica entre bolsa y depósito	Después
Correo de personaje	Entrega en custodia y caducidad	Futuro
Alquiler de herramienta	Contrato y devolución; depósito por estado	Futuro
Taller comunitario	Umbrales de materiales y beneficios temporales	Después
Insignia del fabricante	crafted_by visible y reputación por encargos completos	Después
Receta por fragmentos	Conteo de páginas y consumo al aprender	Después
Feria estacional	Menú temporal, moneda cerrada y epílogo	Futuro
Colección de canciones	Desbloqueos por aventura; audio bajo petición	Después
Ayuda a novatos	Objetivo compartido, recompensa cosmética limitada	Después
Reclutamiento	Tablón con idioma, horario y ritmo	Después
Calendario de grupo	Preferencias y eventos internos optativos	Futuro
Donación a refugios	Contribuciones auditadas; servicios accesibles	Después
Concurso de artesanía	Votación acotada, sin premio de poder	Futuro
Clasificaciones por contribución	Métricas de rescate/artesanía/exploración	Futuro
Cartas de agradecimiento NPC	Derivadas de un evento confirmado	Pronto

Paquete económico recomendado: tres pedidos NPC, uno por herrería, alquimia y cocina. Medir tiempo para conseguir ingredientes, margen y uso del objeto producido. Después permitir que un jugador encargue el mismo objeto a otro.

Evitar: recompensar mensajes repetidos, otorgar confianza por adulación, vender oro o premiar sin límite victorias contra el mismo amigo. Esos sistemas convierten participación social en una fuente de abuso y desdibujan la aventura.

28. Catálogo de ideas adicionales: combate y futuro

Idea	Implementación inicial	Momento
Jefe con foco destructible	SceneObjectState y fase con transición	Después
Batalla de defensa	Objetivo con PV y límite de rondas	Después
Sigilo de grupo	Prueba agregada y coste de fallo definido	Después
Combos de aliados	Evento marca una apertura consumible una vez	Después
Elementos reactivos	Dos interacciones tipadas; límites por ronda	Futuro
Amenaza visible	Prioridades de enemigo anunciadas y protección	Después
Enemigo negociable	Acción social habilitada por fase	Después
Entrenamiento seguro	Encounter de simulación, sin loot real	Pronto
Contrarreloj de ritual	Contador de ronda; victoria sin matar todo	Después
Compañero NPC	Un actor auxiliar con política fija	Futuro
Mascota de clase	Un actor o buff acotado; no enjambres	Futuro
Expedición roguelite	Ruta de elecciones, recursos de instancia	Futuro
Raid de contribución	Aportaciones asíncronas y umbrales	Futuro
Arena de reglas iguales	Equipo normalizado y consentimiento	Futuro
Replay compacto	Mostrar acciones confirmadas y resumen por ronda	Pronto
Tarjeta de héroe	Imagen generada desde datos aprobados	Después
TTS de momentos clave	Caché por hash, voz/idioma y límites	Futuro
Mini App de talentos	Misma API de servicios y initData validada	Futuro
Herramienta de autor	Vista previa YAML, enlaces y simulación	Después
Sandbox de contenido	Versión de catálogo aislada de producción	Después

Junto a las 32 especializaciones, los nuevos linajes, misiones, facciones y ampliaciones de oficio, este catálogo ofrece más de cien propuestas y variantes. Su valor está en elegir conjuntos coherentes. “Jefe ritual + foco destructible + cooperación” es una entrega; activar veinte features sin una aventura que las conecte no lo es.

29. Evidencias y referencias

Método: lectura del árbol y archivos de main fijados al commit indicado; análisis estático de engine, services, ai, bot, schemas, modelos, migraciones y tests; consulta de fuentes públicas; ejecución de pruebas en copia aislada descrita en el anexo. No se accedió a producción ni se usaron claves del usuario. Los PDF históricos binarios en planning/referencias no se extrajeron; sí se revisaron los documentos Markdown actuales. El informe no garantiza balance ni rendimiento de un servicio desplegado.

Fuentes del repositorio

R1. Repositorio y estado:
https://github.com/JD3M0N/Oathforge/tree/31c166f4c2d2e75b8051595f1e5b260f354340f5

R2. Autoridad social: oathforge/ai/npc_dialogue.py, NpcReply; oathforge/services/actions.py, talk; oathforge/services/npcs.py, remember/adjust_trust/steal.

R3. Costes y concurrencia: oathforge/ai/client.py, Budget.allow/record y GeminiLLM._generate; services/chronicles.py, tell; database/models.py, AiUsage.

R4. Identidad y duplicados: services/actions.py, handle_free_text/execute_free_text; bot/handlers/text.py, on_text; services/audit.py, claim/log; bot/handlers/callbacks.py, _act y callbacks de combate.

R5. Cooperativo: services/combat_service.py, active_encounter/start_encounter/act/_sync_and_maybe_finish; engine/combat.py, player_act/_run_enemies; database/models.py, PartyMember/EncounterParticipant.

R6. Persistencia: services/activities.py, start/claim; bot/handlers/common.py, schedule_ready_ping/_ready_ping; recorder.py. Revisar líneas por función: números cambian al editar.

Fuentes externas consultadas el 14/10/2026

S1. Wizards, D&D Free Rules 2024, Playing the Game:
https://www.dndbeyond.com/sources/dnd/br-2024/playing-the-game

S2. Blizzard, Dragonflight: Making It with Professions:
https://worldofwarcraft.blizzard.com/en-us/news/23876529/dragonflight-making-it-with-professions

S3. Blizzard, The War Within: Warbands Preview:
https://worldofwarcraft.blizzard.com/en-us/news/24061008/the-war-within-warbands-preview (resumen indexado; algunos artículos de Blizzard devolvieron 403 al abrir).

S4. Chat Wars, canal oficial de novedades: https://t.me/s/chatwars (distingue cambios de producción y staging; no se ha confirmado despliegue posterior de cada anuncio).

S5. SimpleMMO, presentación oficial: https://simple-mmo.com/about

T1. Telegram Bot API: https://core.telegram.org/bots/api

T2. SQLAlchemy, Configuring a Version Counter:
https://docs.sqlalchemy.org/en/20/orm/versioning.html

T3. PostgreSQL, Explicit Locking: https://www.postgresql.org/docs/current/explicit-locking.html

Las reglas, nombres e imágenes de los referentes no se incluyen como contenido para copiar. Usar las inspiraciones como patrones y desarrollar expresiones propias para Aetherfall.

30. Anexo: verificación ejecutada

Resultado final: 287 pruebas aprobadas y 2 omitidas en 18,73 segundos. Ejecución: python -m pytest -q, sobre copia aislada de la versión 0.7.6, Python 3.12.14 y dependencias instaladas desde pyproject.toml. No se modificaron reglas del proyecto para hacer pasar las pruebas. Los servicios externos se simularon mediante los fakes de la suite.

La primera ejecución sobre una copia de texto incompleta produjo 278 aprobadas, 9 fallidas y 2 omitidas: los nueve fallos requerían imágenes no materializadas todavía. Se recuperaron los recursos del mismo commit y se repitió la suite completa; esos fallos desaparecieron. Por tanto, no se presentan como bugs de Oathforge.

Dos pruebas omitidas: concurrencia de compra y de reclamación de actividad, porque el fixture usa SQLite cuando PostgreSQL no está disponible. No se verificaron aquí locks reales de PostgreSQL, carreras económicas de producción, despliegue del bot, cuotas reales de Gemini ni latencia de Telegram.

La suite aprobada respalda la funcionalidad cubierta por los tests existentes. Los hallazgos de los capítulos 21-22 siguen siendo lectura estática y recomendaciones de pruebas adicionales; no se afirma haber reproducido todos en un servidor multiusuario. El primer ticket del Roadmap propone cerrar precisamente esa diferencia de cobertura.

Detalle de ambiente: instalación editable con las dependencias permitidas por sus rangos; no existía un lock de dependencias en el árbol revisado. Eso permite repetir una instalación, pero no garantiza obtener idénticas versiones en otra fecha. Añadir lock y matriz CI es parte de la propuesta.